Adaptation of Visual Reasoners Based on Code Generation Through Fine-Tuning and Preference Optimization

Josu Barrutia

7 July 2025

Faculty of Informatics, UPV/EHU

Outline

Visual Reasoning

Visual Reasoning

Ability to understand actions and reasoning associated with any visual images.

Query: how many muffins can each kid have for it to be fair?

Input Image:

End-to-end models

Models trained to map an image and a textual query directly to an answer.

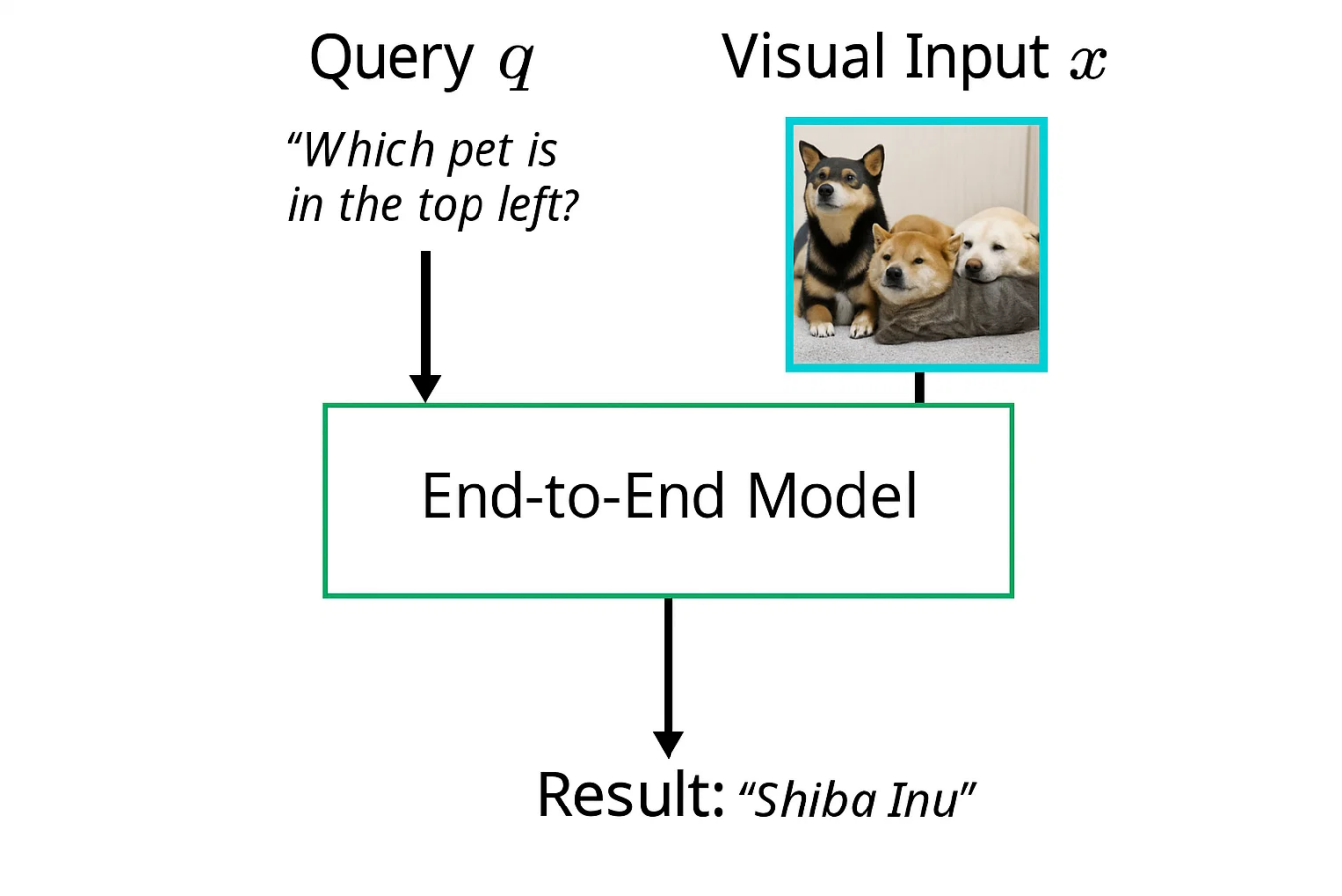


Image generated with Sora

End-to-end models

Limitations:

- Tokens are discrete, the world is continuous
- They do not inherently leverage compositional reasoning (uninterpretable)
- Poor generalization

End-to-end models

Limitations:

- Tokens are discrete, the world is continuous
- They do not inherently leverage compositional reasoning (uninterpretable)
- Poor generalization

End-to-end models

Limitations:

- Tokens are discrete, the world is continuous
- They do not inherently leverage compositional reasoning (uninterpretable)
- Poor generalization

ViperGPT

Query: how many muffins can each kid have for it to be fair?

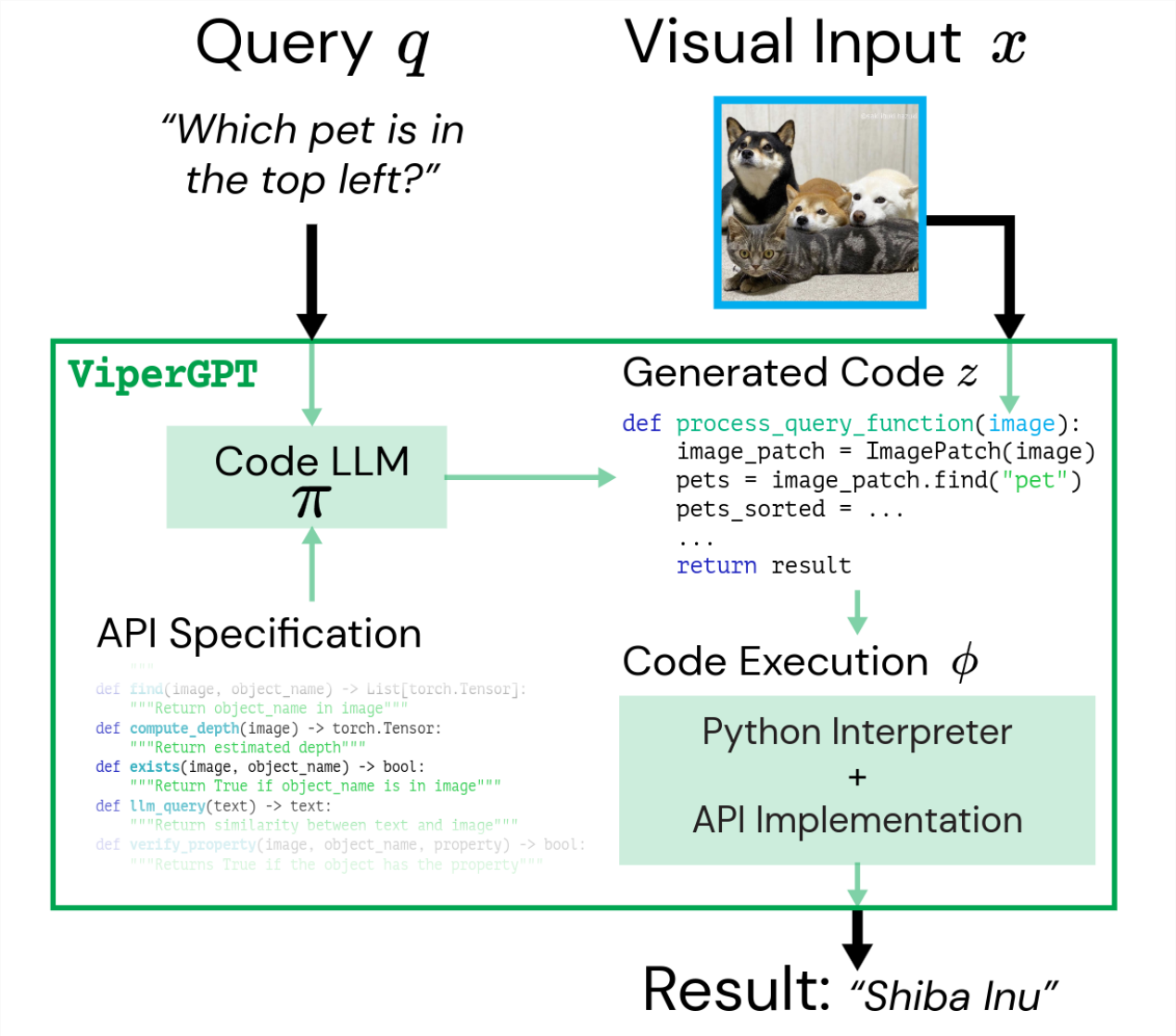
Input Image:

Figure 1: ViperGPT: Visual Inference via Python Execution for Reasoning [3]

ViperGPT

Input:	x (visual input), q (textual query)
Program generation:	$z = \pi_\theta(q)$
Execution:	$r = \phi(x, z)$

ViperGPT


Query q
"Which pet is in the top left?"
Visual Input x
ViperGPT
Code LLM π
API Specification
def find(image, object_name) -> List[torch.Tensor]:
"""Return object_name in image"""
def compute_depth(image) -> torch.Tensor:
"""Return estimated depth"""
def exists(image, object_name) -> bool:
"""Return True if object_name is in image"""
def llm_query(text) -> text:
"""Return similarity between text and image"""
def verify_property(image, object_name, property) -> bool:
"""Returns True if the object has the property"""
Generated Code z
def process_query_function(image):
image_patch = ImagePatch(image)
pets = image_patch.find("pet")
pets_sorted = ...
...
return result
Code Execution ϕ
Python Interpreter
+
API Implementation
Result: "Shiba Inu"

ViperGPT

Query:
Is the folding chair to the right of the boy white or red?

Input:

ViperGPT Reasoning:

```
1  def execute_command(image):
2    image_patch = ImagePatch(image)
3    boy_patches = image_patch.find("boy")
4    for boy_patch in boy_patches:
5      folding_chair_patches = image_patch.find("folding ch[...]
6      for folding_chair_patch in folding_chair_patches:
7        if folding_chair_patch.horizontal_center > boy_p[...]
8          return folding_chair_patch.best_text_match("[...]
9    return "not found"
```

ViperGPT Execution:

folding_chair_patches:

This call is handled by GLIP

ViperGPT

Query:

What kind of clothing is brown?

Input:

ViperGPT Reasoning:

```
1  def execute_command(image):
2    image_patch = ImagePatch(image)
3    clothing_patches = image_patch.find("clothing")
4    for clothing_patch in clothing_patches:
5      if clothing_patch.verify_property("clothing", "brown[...]
6        return clothing_patch.simple_query("What kind of[...]
7    return "no clothing is brown"
```

ViperGPT Execution:

return:
suit

This call is handled by BLIP

ViperGPT

Query:

person in sunglasses

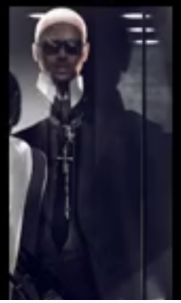
Input:

ViperGPT Reasoning:

```
1   def execute_command(image):
2     image_patch = ImagePatch(image)
3     person_patches = image_patch.find("person")
4     person_patch = best_image_match(person_patches, ["sungla[...]
5     return person_patch
```

ViperGPT Execution:

person_patch:



This call is handled by X-VLM

ViperGPT

Is it possible to adapt the program generator to a specific task?

Problem:

$$r = \phi(x, \pi_\theta(q)) \quad \text{is not differentiable w.r.t. } \theta$$

ViperGPT

Composition: $r = \phi(x, \pi_\theta(q))$

Issue:

- $\pi_\theta(q)$ is sampled and then passed to $\phi(x, z)$
- This composition is **not differentiable** w.r.t. θ

Implication:

- Cannot train π_θ with standard backpropagation
- Ground-truth programs z for supervision are typically unavailable

ViperGPT

ViperGPT's Solution:

- Use Codex (code-davinci-002) as a program generator
- Generate **Python code** directly from the query q
- Prompted with:
  - An API of vision modules
  - Few-shot examples
  - The current query

Benefit: LLM has already been trained at scale on Python code from the internet

ViperGPT

Observation:

- We can compare the output $r = \phi(x, \pi_\theta(q))$ to the ground-truth answer

Key Insight:

- **Correct answer** $\Rightarrow$ likely **good reasoning code**
- **Wrong answer** $\not\Rightarrow$ necessarily bad code (could be a VLM failure)

This thesis: We leverage this signal to improve the visual reasoning behavior of the LLM program generator.

GQA Dataset

GQA is a benchmark for compositional visual reasoning [1].

We work exclusively with the **balanced version** of the dataset, which is split into:

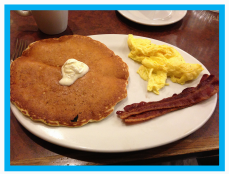
- **Training Set**
  - 943000 questions total
  - Only **first 12600** used for dataset creation
- **Validation Set**
  - 132062 questions total
  - Only **first 1000** used for hyperparameter search during training
- **Testdev Set**
  - 12578 questions
  - Used for **final model evaluation**

GQA Dataset

GQA questions often require multi-step, compositional reasoning.

Query: Does that pancake look brown and round?

In:



Generated code

```
def execute_command(image):
    image_patch = ImagePatch(image)
    pancake_patches = image_patch.find("pancake")
    is_brown = pancake_patches[0].verify_property("pancake", "brown")
    is_round = pancake_patches[0].verify_property("pancake", "round")
    return bool_to_yesno(is_brown and is_round)
```

Execution

```
pancake_patches = image_patch.
      find("pancake")
▶ pancake_patches[0] = {ImagePatch}
```

```
...verify_property("pancake", "brown")
▶ is_brown = {bool} True
...verify_property("pancake", round)
▶ is_round = {bool} True
▶ is_brown and is_round = {bool} True
Result: "yes"
```

Query: Are there water bottles to the right of the bookcase that is made of wood?

In:

Generated code

```
def execute_command(image):
    image_patch = ImagePatch(image)
    bookcase_patches = image_patch.find("bookcase")
    for bookcase_patch in bookcase_patches:
        is_wood = bookcase_patch.verify_property("bookcase", "wood")
        if is_wood:
            water_bottle_patches = image_patch.find("water bottle")
            for water_bottle_patch in water_bottle_patches:
                if water_bottle_patch.horizontal_center > \
                   bookcase_patch.horizontal_center:
                    return "yes"
            return "no"
    return "no"
```

Execution

```
bookcase_patches= image_patch.
      find("bookcase")
▶ bookcase_patches[0] = {ImagePatch}
```

```
▶ bookcase_patches[0].
     horizontal_center = {float} 239.0
...verify_property("bookcase","wood")
▶ is_wood = {bool} True
```

```
water_bottle_patches = image_patch.
      find("water bottle")
▶ water_bottle_patches[0]
   = {ImagePatches}
```

```
▶ water_bottle_patches[0].
     horizontal_center = {float} 608.5
▶ water_bottle_patch.horizontal_center >
bookcase_patch.horizontal_center =
{bool} True
```

Result:"yes"

Adaptation Methods

Adaptation Methods

Goal: Adapt the program generator to produce better visual reasoning code

Two Adaptation Strategies:

- Supervised Fine-Tuning (SFT)
- Direct Preference Optimization (DPO)

Supervised Fine-Tuning (SFT)

Objective: Adapt LLMs using correct code completions for visual queries

What do we need?

- A dataset of **ground-truth code sequences**

Best used with:

- Instruction-tuned models (already trained to follow prompts)
- Special tokens: `[SOS]`, `[EOS]`, `[SEP]`, etc.

SFT Objective Function

Dataset: $D = \{(x_i, y_i)\}_{i=1}^{N}$ where:

- x_i: prompt (query)
- y_i: ground-truth code (target)

Loss:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{(x,y)\in D}\sum_{j=1}^{|y|}\log P(y_j \mid y_{<j}, x; \theta)$$

Training strategy: Teacher Forcing

- Use ground-truth tokens $y_{<j}$ as input
- Predict next token y_j

Masking the Prompt During SFT Loss

Training instance:

`[API] [Few-Shot] [Current Query] [Answer Code]`

Loss is only computed on:

`[Answer Code]`

Only the answer code tokens contribute to the loss. The rest are masked.

Direct Preference Optimization (DPO)

Objective: Fine-tune the LLM to prefer better code completions [2]

What do we need?

- A dataset of **preference pairs** for each query:
  - Preferred: y_w
  - Rejected: y_l

Increase the likelihood of y_w and reduce the likelihood of y_l

DPO Preference Pair Example

Prompt Structure:

`[API] [Few-Shot] [Current Query]`

Two possible completions:

Preferred Completion (y_w)

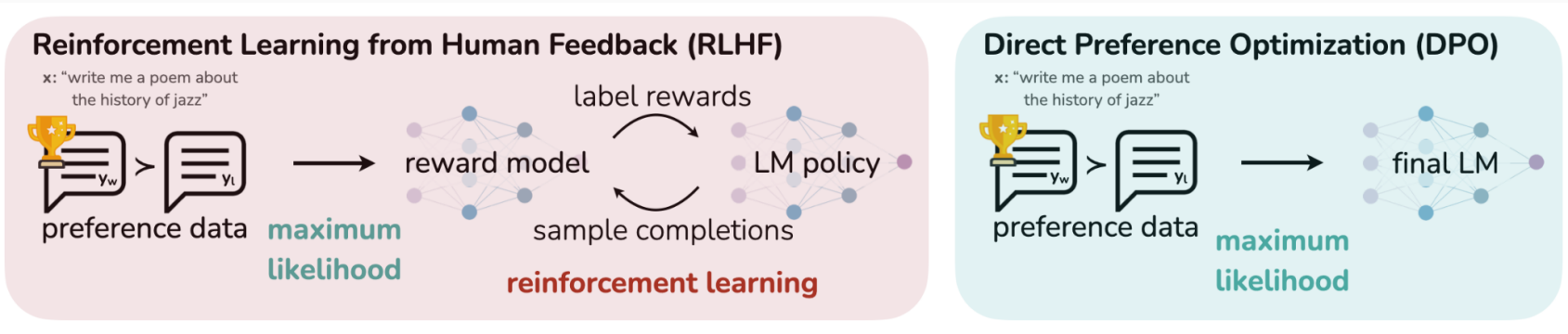
[Preferred Code]

Rejected Completion (y_l)

[Rejected Code]

DPO optimizes the model to prefer the better code.

Direct Preference Optimization (DPO)



No reward model or reinforcement learning needed.

DPO Loss Function

Given:

- Prompt x
- Preferred completion y_w, Rejected completion y_l
- Current policy π_θ, Reference policy π_{ref}

Loss:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)\right)\right]$$

$$\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$$

Interpretation:

- Increase preference for y_w, decrease for y_l
- β: controls the deviation from the base reference policy π_{ref}

Dataset creation

Dataset Creation

Goal: Create a dataset of code completions for visual queries

Steps:

- Generate code completions using multiple LLMs
- Execute the generated code to evaluate correctness
- Classify the completions into distinct error categories

Ideal Preference Hierarchy

Ideal preference hierarchy with five distinct reasoning-quality levels.

Practical Preference Hierarchy (Used in Practice)

Practical preference hierarchy used by the automatic classifier.

LLMs Used

LLM	Syntax Error	Runtime Error	Semantic/Inference Error	Correct
Llama3.1-8B-It	155	199	6018	6228
CodeLlama-7B-hf-It	193	997	6476	4934
Mixtral-8x7B-It	295	1646	5802	4857
DeepSeek-Llama8B	1152	1007	5658	4783
Qwen2.5-Math-7B	2089	1976	5057	3478
DeepSeek-Qwen7B	3018	2026	4480	3076

Each model generated 12,600 code completions on the GQA training split.
Executions evaluated using GLIP-Large and BLIP2-Flan-T5-XL.

Instance Patterns Across the Dataset

Pattern	Compilation Error	Runtime Error	Semantic or Inference Error	Correct	# instances
A				✓	443
B			✓	✓	2209
C		✓	✓	✓	1973
D	✓	✓	✓	✓	1198
E	✓		✓	✓	1820
F		✓		✓	447
G	✓	✓		✓	375
H	✓			✓	409
I			✓		793
J		✓	✓		1078
K	✓	✓	✓		846
L	✓		✓		1008
M		✓			0
N	✓	✓			1
O	✓				0
				Total	12600

Table 1: Group each visual question instance into one of 15 patterns based on the classification of the generated code accross the 6 models.

Two types of datasets

We have 6 different code reasoning strategies over each visual query, for 12600 instances.

With these codes, we generate:

- Preference dataset: two variants
- SFT dataset

From Reasoning Hierarchy to Preference Dataset

Recall: We have a strict hierarchy for reasoning quality:

$$\texttt{Correct} > \texttt{Semantic/Inference} > \texttt{Runtime} > \texttt{Syntax}$$

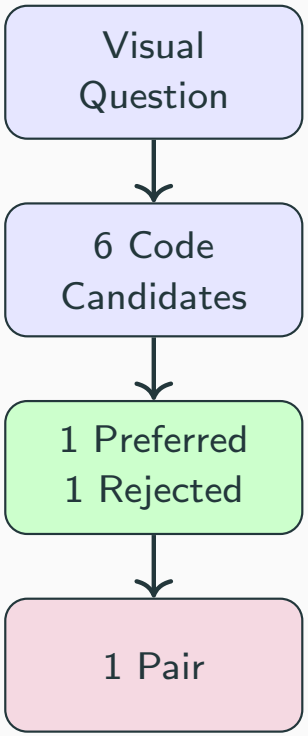
Key idea: For any visual question that falls into patterns B-H, we can compare a correct code with a lower quality code

This allows us to construct:

- A dataset of **preferred** vs **rejected** codes
- We will select a code classified as correct as preferred and a code falling into a lower category as rejected

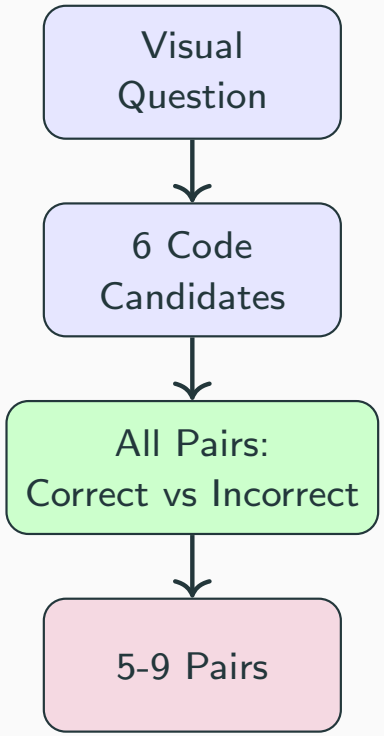
Two Variants of the Preference Dataset

Single-Pair Dataset



7431 Train / 1000 Dev Instances

All-Pair Dataset



52487 Train Instances

Constructing the SFT Dataset

Goal: Collect code completions that correctly answer the visual question

Selection Criteria:

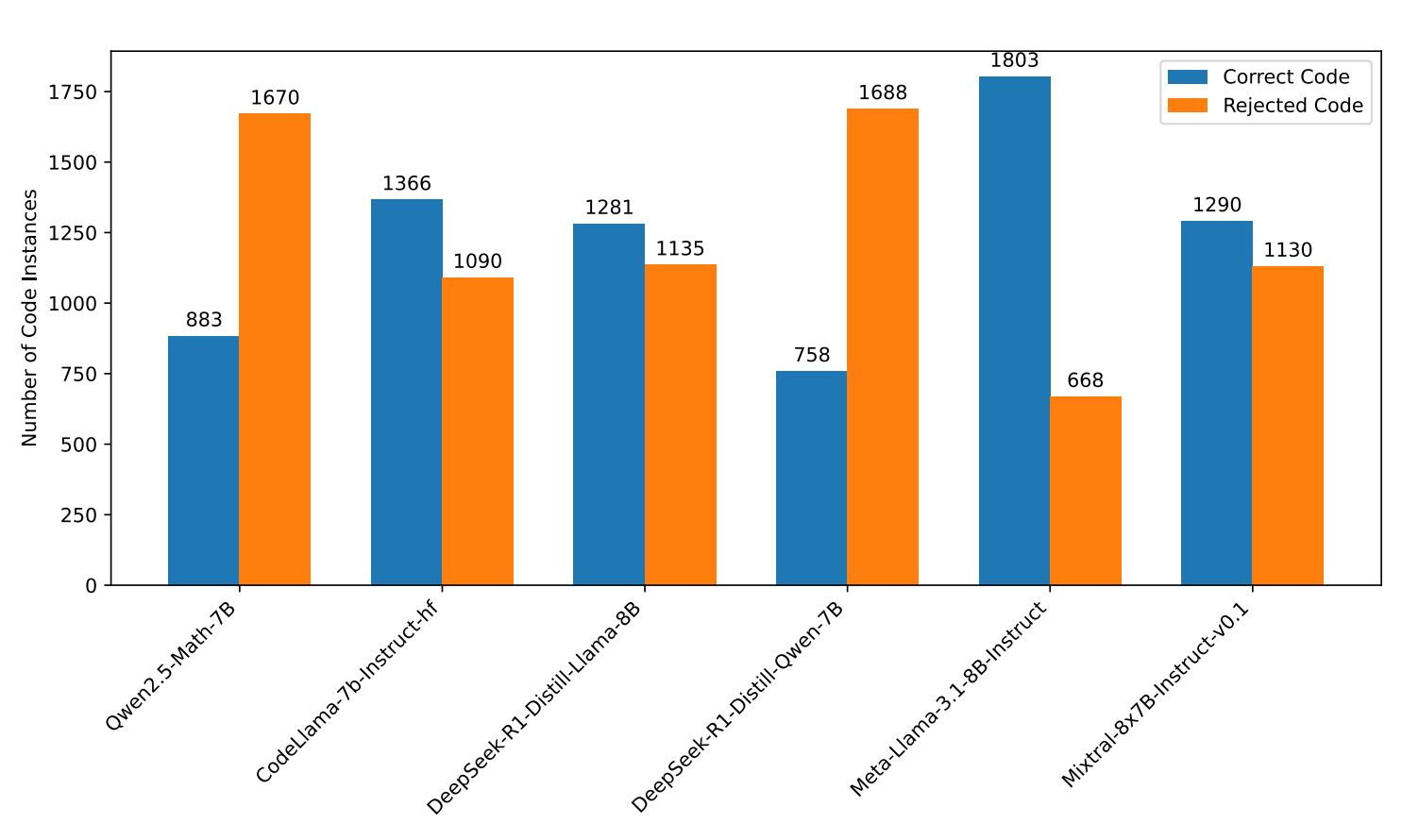
- Use instances from patterns A-H (at least one correct code exists)
- Randomly select one **correct** program per instance
- Exclude the same 1000 dev instances excluded from the single-pair dataset to avoid contamination

Final Result:

- **8874** question-code pairs total
- **7874 train** / **1000 dev**

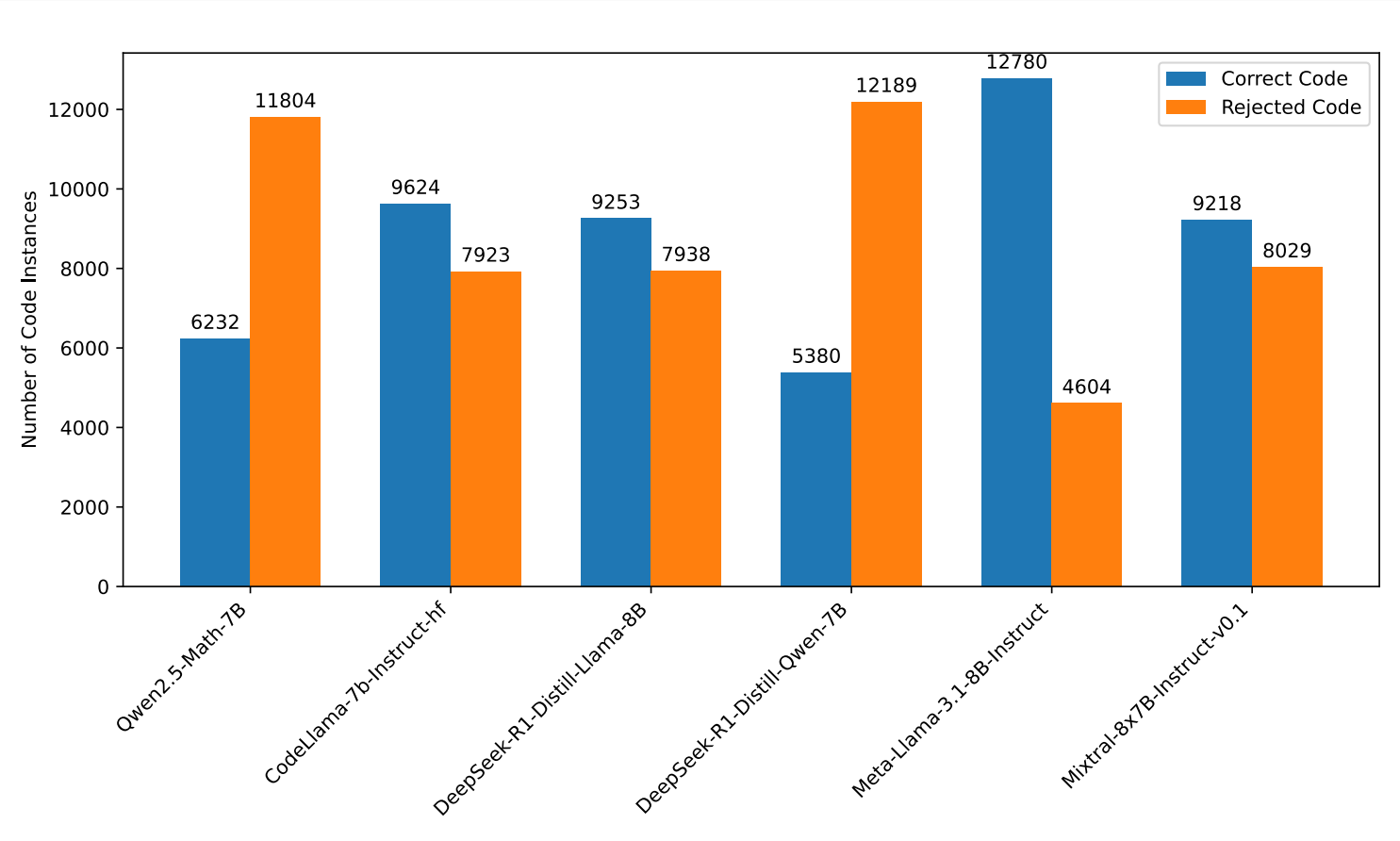
Model Distribution

Which models produced preferred vs rejected code?



Model distribution for the Single-pair Preference Dataset.

Model Distribution

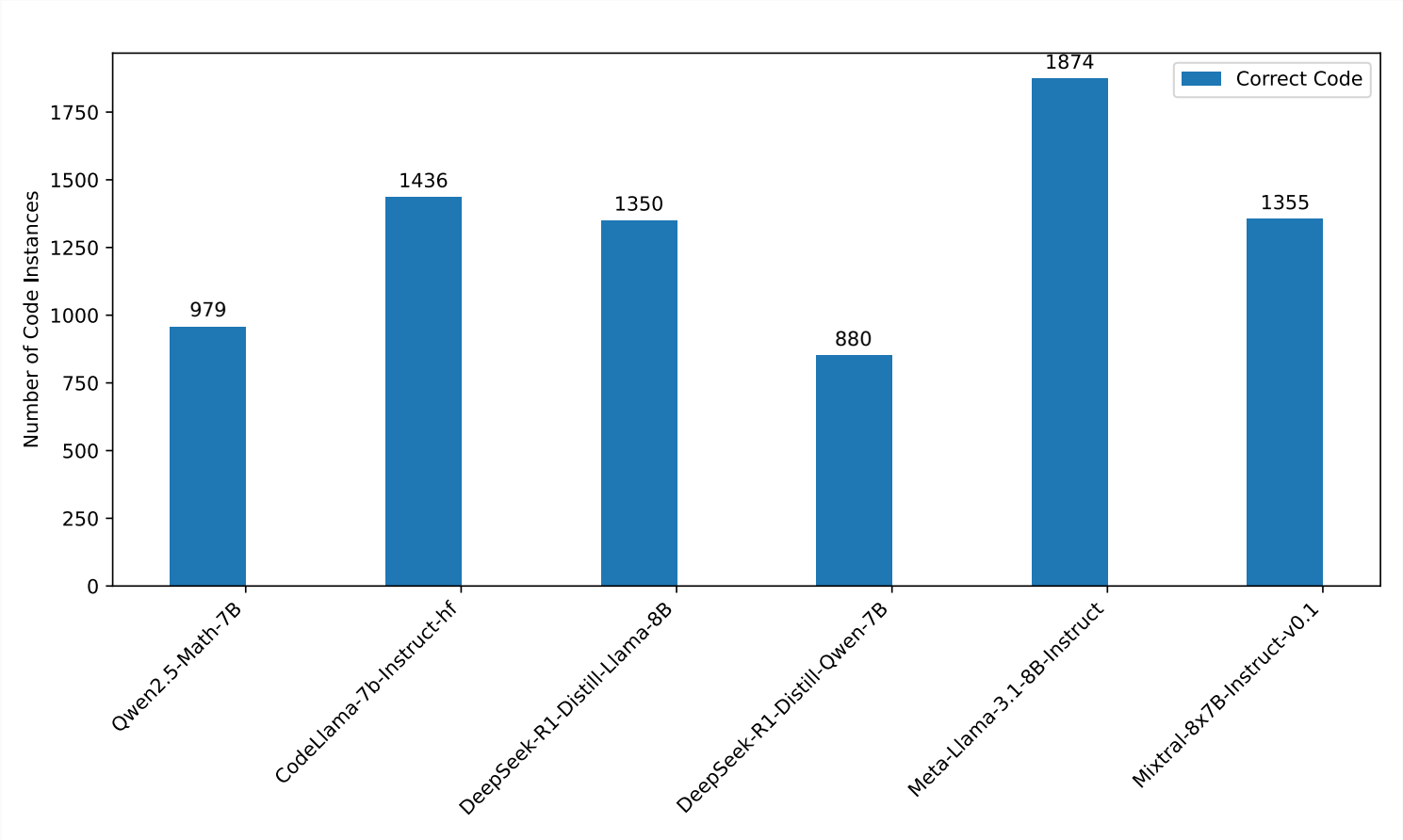


Model distribution for the All-pair Preference Dataset.

Model Distribution

Which models contributed the most correct code?

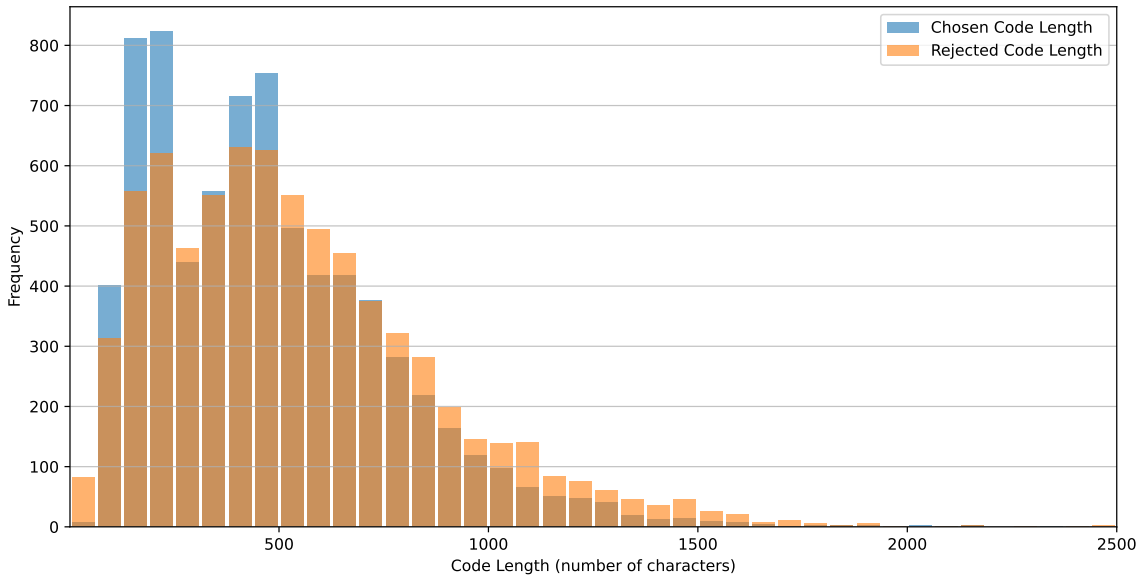
- Distribution mirrors what we saw in the preference dataset.



Model distribution for the SFT Dataset.

Code Length Distribution (Single-Pair Preference Dataset)

Do better codes tend to be longer? Not always.



Key insight:

- Preferred codes tend to be **shorter**, unless too short (trivial).
- Shorter programs often reflect simpler, more effective reasoning paths.
- Longer programs may involve more VLM calls, increasing failure risk.

Results

DPO Performance Summary

LLM	Syntax Error ↓	Runtime Error ↓	Semantic / Inference Error ↓	Correct ↑
Not Fine-Tuned (Baseline)				
ViperGPT	-	-	-	48.10
Qwen2.5-Math-7B	0.78	7.56	52.33	39.33
CodeLlama-7B-hf	0.78	0.60	55.61	43.02
Llama-3.1-8B	1.19	0.39	53.29	45.13
Fine-Tuned Models				
Qwen2.5-Math-7B	0.05	8.79	49.62	41.55 (+2.22)
CodeLlama-7B-hf	0.19	0.17	53.10	46.53 (+3.51)
Llama-3.1-8B	0.13	0.10	51.98	47.80 (+2.67)

Table 2: Comparison of zero-shot (including ViperGPT) and DPO fine-tuned LLMs on the GQA testdev partition. To evaluate our models GLIP-Large and BLIP2-Flan-T5-XL were used.

SFT Performance Summary

LLM	Syntax Error ↓	Runtime Error ↓	Semantic / Inference Error ↓	Correct ↑
Not Fine-Tuned (Baseline)				
ViperGPT	-	-	-	48.10
Qwen2.5-7B-It	2.71	3.12	52.72	41.45
CodeLlama-7B-It-hf	0.06	1.14	54.79	44.01
Llama-3.1-8B-It	0.12	0.89	54.45	44.54
Fine-Tuned Models				
Qwen2.5-7B-It	0.01	0.06	53.67	46.27 (+4.82)
CodeLlama-7B-It-hf	0.02	0.05	53.03	46.90 (+2.89)
Llama-3.1-8B-It	0.03	0.14	52.92	46.92 (+2.38)

Table 3: Comparison of zero-shot (including ViperGPT) and supervised fine-tuned LLMs on the GQA testdev partition. To evaluate our models GLIP-Large and BLIP2-Flan-T5-XL were used.

Analysis

Comparison with ViperGPT

ViperGPT

- code-davinci-002 (175B params)
- GLIP-L (large)
- X-VLM (MSCOCO retrieval)
- BLIP-2 Flan-T5-XXL
- Testdev acc.: 48.1%

Our Approach

- Llama3.1-8B$_{\text{DPO}}$ (8B params)
- GLIP-L (large)
- X-VLM (MSCOCO retrieval)
- BLIP-2 Flan-T5-XL
- Testdev acc.: 47.8%

Semantic and inference errors estimation

We performed a manual evalution over 40 instances from results of Llama3.1-8B$_{\text{DPO}}$ on the GQA testdev partition:

- Inference failures: **90%**
- Semantic (logic) errors: **10%**
- *Of all samples, 60% used non-compositional reasoning*

Non-Compositional Reasoning Example

"What color is the dirt?"

Non-Compositional Reasoning Example

```
def execute_command(image):
    im_patch = ImagePatch(image)
    return im_patch.simple_query("What␣color␣is␣the␣dirt?")
```

Relies solely on BLIP-2 XL (44.2% acc.)

- Answer: white
- Ground-truth: red

Compositional Reasoning Example

"Is there an elephant near the person that is wearing a coat?"

Compositional Reasoning Example

```
def execute_command(image):
    im_patch = ImagePatch(image)
    el_patches = im_patch.find(""elephant"")
    if len(el_patches) == 0:
        return im_patch.simple_query(""Is there an elephant near the person that
     is wearing a coat?"")
    for el_patch in el_patches:
        per_patches = im_patch.find(""person with coat"")
        if len(per_patches) == 0:
            return ""no""
        for per_patch in per_patches:
            if el_patch.horizontal_center > im_patch.horizontal_center / 2:
                if per_patch.horizontal_center > im_patch.horizontal_center / 2:
                    return ""yes""
    return ""no""
```

Despite correct steps, VLM misprediction leads to wrong answer.

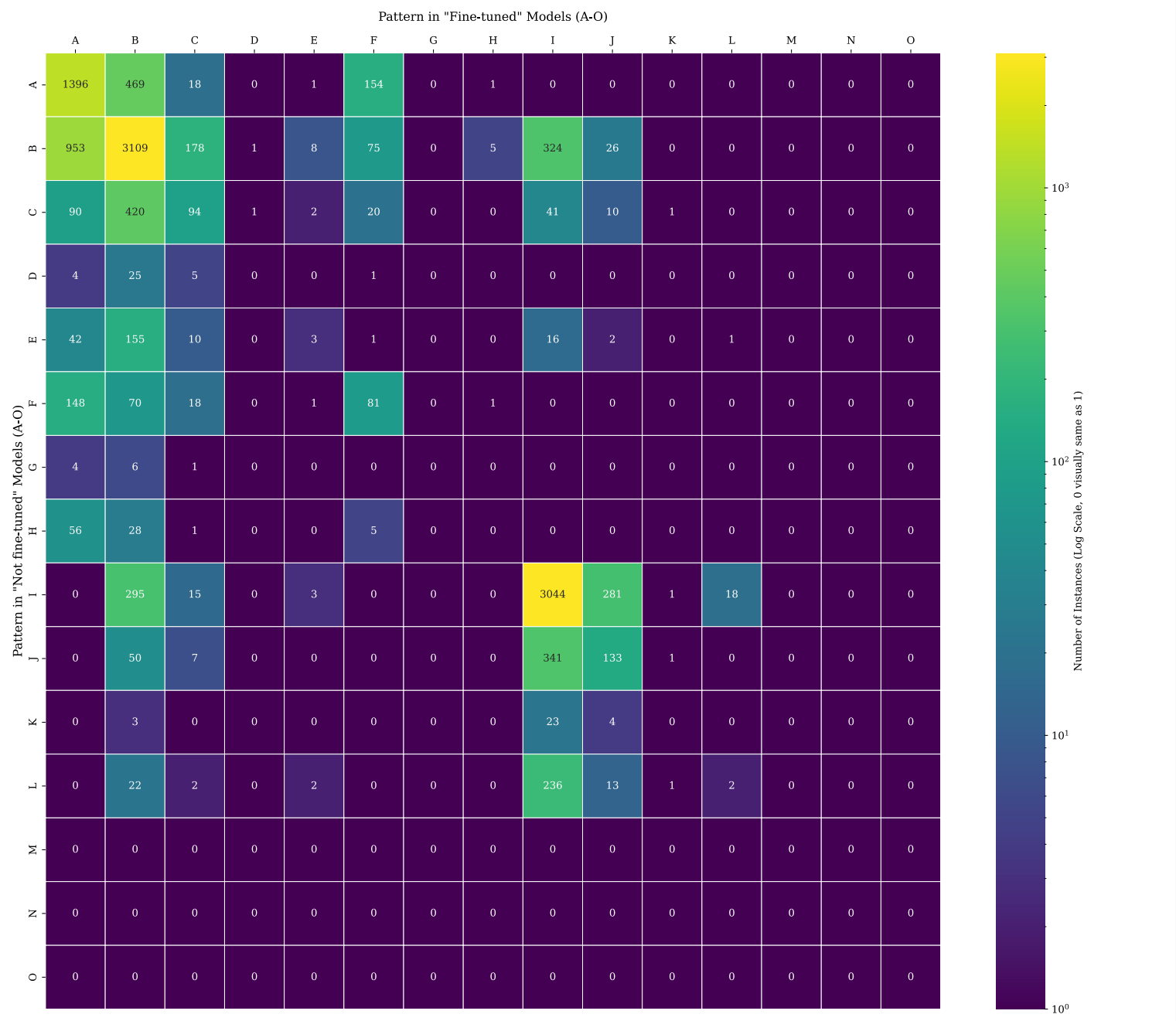
- Answer: yes
- Ground-truth: no

Collective vs Individual Knowledge

Pattern	Compilation Error	Runtime Error	Semantic or Inference Error	Correct	Instances (Not fine-tuned)	Instances (Fine-tuned)
A				✓	2039	2693
B			✓	✓	4679	4652
C		✓	✓	✓	679	349
D	✓	✓	✓	✓	35	2
E	✓		✓	✓	230	20
F		✓		✓	319	337
G	✓	✓		✓	11	0
H	✓			✓	90	7
I			✓		3656	4024
J		✓	✓		532	469
K	✓	✓	✓		30	4
L	✓		✓		278	21
M		✓			0	0
N	✓	✓			0	0
O	✓				0	0
				Total	12578	12578

Table 4: Distribution of the 12578 testdev visual question instances across patterns in the not fine-tuned and fine-tuned model evaluations.

Collective vs Individual Knowledge



Collective vs Individual Knowledge

Instance Transitions After Training:

- **Improved patterns ($i > j$):** 3068 instances
- **Worsened patterns ($i < j$):** 2078 instances
- **Unchanged: ($i = j$):** remaining instances

Color Group Changes (Generalization):

- Example: 295 moved from yellow (I) to green (B)
- But there are also examples of green to yellow transitions

Collective vs Individual Knowledge

Collective Accuracy (A-H):

- Before training: $8082/12578 \approx 64.2\%$
- After training: $8060/12578 \approx 64.1\%$

Fine-tuning goal is to bring each model's individual knowledge closer to the collective knowledge of the group. But we hoped generalization could implicitly occur.

Future Work

- Iterative data refinement
- Combining SFT and DPO
- Scaling the training dataset
- Scaling model diversity and size
- Beyond LoRA: Full Fine-Tuning

Conclusions

Main Contributions:

- Constructed two synthetic datasets from GQA:
  - **SFT dataset**
  - **DPO dataset:** two variants
- We showed that LLMs visual reasoning capabilities can be improved with DPO and SFT

Key Findings:

- All fine-tuned models showed improvements of at least 2% accuracy, and up to nearly 5%, over the baselines
- We reached up to **47.80% accuracy**, nearly matching ViperGPT's 48.1%
- Training consolidated **collective knowledge** into single models, but no new generalization was observed

Conclusions

This thesis shows that program-based visual reasoners can be trained, not just prompted.

Thank you for your attention!

Questions, comments, or ideas?

jbarrutia006@ikasle.ehu.eus @github/JosuBarru

References i


Drew A Hudson and Christopher D Manning.
Gqa: A new dataset for real-world visual reasoning and compositional question answering.
In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709, 2019.


Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn.
Direct preference optimization: Your language model is secretly a reward model, 2024.


Dídac Surís, Sachit Menon, and Carl Vondrick.
Vipergpt: Visual inference via python execution for reasoning, 2023.